



UNIVERSITÀ DEGLI STUDI DI FIRENZE

Scuola di Ingegneria
Corso di Laurea in Ingegneria Informatica

ChargeNet

Piattaforma per la gestione di stazioni di ricarica EV

Candidati:

Tommaso Sottili 7138541
Luca Moracci 7140812
Marco Lampronti 7147400

Corso:

Ingegneria del software

Docente:

Prof. Enrico Vicario

Anno Accademico 2025/2026

Indice

1	Introduzione	2
1.1	Descrizione	2
1.2	Stack Tecnologico	2
1.3	Astrazione dell'Infrastruttura Fisica	3
2	Analisi dei Requisiti e Casi d'uso	4
2.1	Use Case Diagram	4
2.2	Use Case Templates	5
2.3	Mockups	9
2.4	Page Navigation Diagram	9
2.5	Package Diagram	9
2.6	Class Diagram	10
2.7	ER Diagram e Modello Relazionale	10
3	Implementazione	11
3.1	Domain Model	11
3.1.1	Utenti	12
3.1.2	Infrastrutture	12
3.1.3	Sessioni	13
3.1.4	Finanze	14
3.1.5	Feedback	15
3.2	Business Layer	16
3.2.1	GridCluster	16
3.2.2	GridMonitor	17
3.2.3	LoadBalancerService	17
3.2.4	SessionService	18
3.2.5	StationService	19
3.2.6	Sicurezza e Validazione	19
3.2.7	Gestione Utente e Portafoglio	20
3.2.8	Gestione Feedback e Manutenzione	20
3.3	Controller (Application Layer)	22
3.4	Design Pattern Implementati	22
4	Testing	23
5	Demo	24

Capitolo 1

Introduzione

1.1 Descrizione

Il progetto **ChargeNet** è un sistema software pensato per la gestione di una rete di stazioni di ricarica per veicoli elettrici (EV). L'idea alla base dell'applicazione è quella di coordinare tutto ciò che ruota attorno a una sessione di ricarica: dall'aiutare l'utente a trovare una colonnina libera, fino alla gestione del pagamento, assicurandosi allo stesso tempo che l'infrastruttura elettrica non subisca danni.

Per raggiungere questo obiettivo, la piattaforma fa da punto di incontro per tre specifiche figure, fornendo a ciascuna le funzionalità necessarie:

- **Driver:** È l'utente finale. Utilizza il sistema per cercare le stazioni compatibili con la sua auto, far partire e monitorare la ricarica, e gestire i pagamenti tramite un portafoglio virtuale interno (wallet).
- **Station Operator:** È l'addetto alla manutenzione fisica. Il suo compito è controllare lo stato di salute delle colonnine: se una stazione si guasta, può metterla temporaneamente fuori servizio per evitare disagi ai Driver, e registrarla di nuovo come attiva una volta riparata.
- **Energy Manager:** È il tecnico supervisore della rete elettrica. Controlla in tempo reale il carico sui trasformatori per evitare surriscaldamenti. Se la rete è sotto stress, ha gli strumenti per abbassare la potenza erogata o limitare le ricariche (Load Balancing) in modo da proteggere l'infrastruttura.

1.2 Stack Tecnologico

Per lo sviluppo è stato scelto un ecosistema Java-centrico, privilegiando la robustezza e la portabilità:

- **Linguaggio:** Java 21.
- **Interfaccia Grafica:** JavaFX 21, per una gestione reattiva delle dashboard dei vari attori.
- **Persistenza:** PostgreSQL, gestito tramite JDBC per un controllo granulare delle transazioni.
- **Build Tool:** Maven, per la gestione delle dipendenze e del ciclo di vita del software.

1.3 Astrazione dell'Infrastruttura Fisica

Trattandosi di un progetto universitario, l'applicazione non comunica con colonnine e trasformatori reali. Nel mondo reale, infatti, interfacciarsi con l'hardware fisico è un'operazione che porta con sé diverse complicazioni: bisogna gestire le differenze tra i vari produttori, adattarsi a protocolli di rete specifici e gestire i fisiologici ritardi o cadute di connessione dei sensori.

L'architettura del sistema è stata comunque pensata per un'eventuale integrazione futura. Per le finalità di questo progetto, i dati sui consumi e sullo stato delle stazioni vengono semplicemente simulati da un componente interno al programma (*GridMonitor*).

Capitolo 2

Analisi dei Requisiti e Casi d'uso

Questo capitolo descrive il processo di progettazione del sistema, seguendo un approccio che va dall'esterno verso l'interno. Si parte dall'analisi di come gli utenti interagiscono con l'applicazione, definendo i requisiti e l'interfaccia grafica, per poi scendere progressivamente nei dettagli tecnici dell'architettura software e della struttura del database.

2.1 Use Case Diagram

Il diagramma dei casi d'uso offre una visione d'insieme delle funzionalità del sistema. Serve a mappare visivamente quali azioni possono compiere i tre attori principali (Driver, Station Operator ed Energy Manager) all'interno della piattaforma.

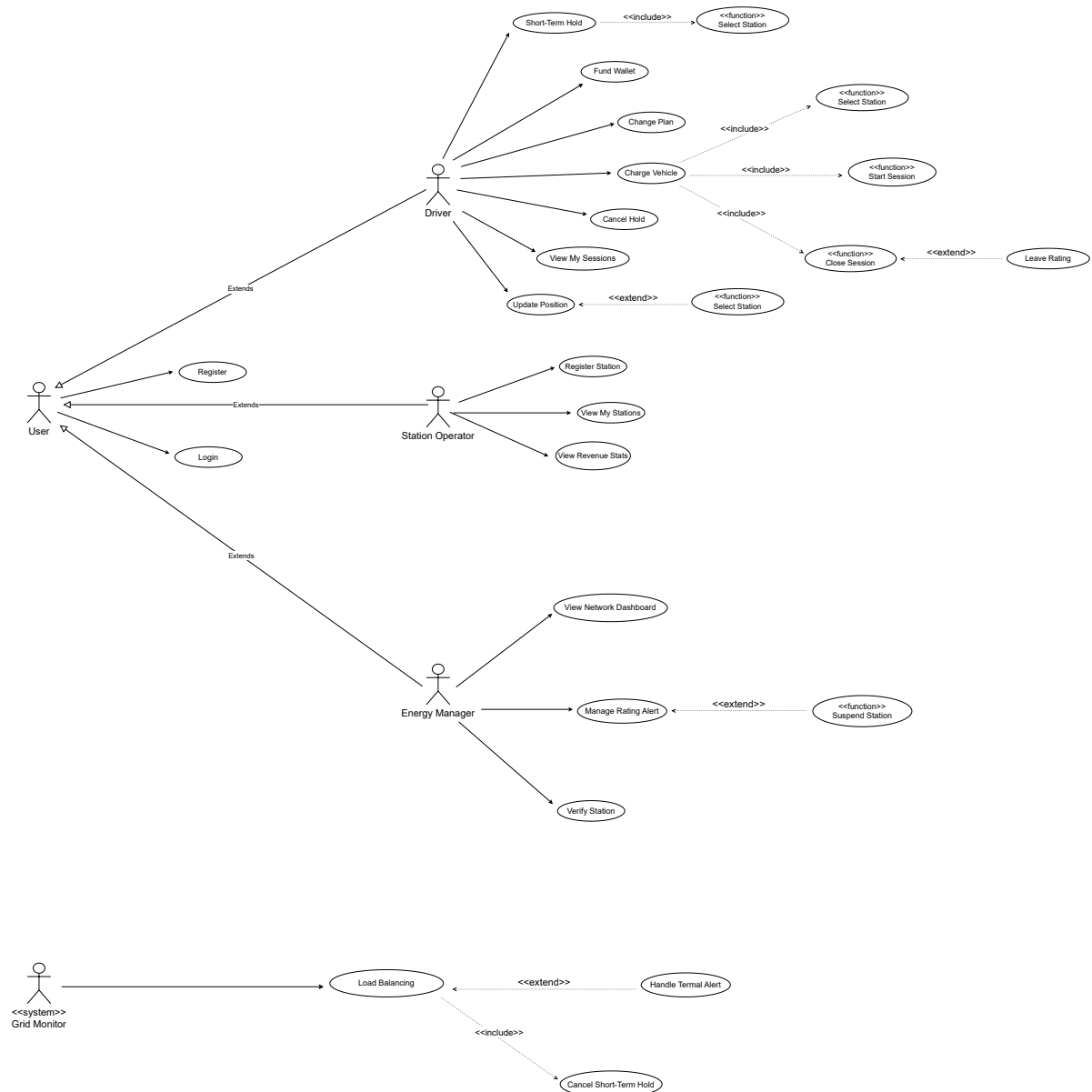


Figura 2.1: Diagramma dei casi d'uso

2.2 Use Case Templates

Per evitare ambiguità, i casi d'uso più importanti sono stati documentati nel dettaglio attraverso dei template strutturati. Questi schemi descrivono passo dopo passo il flusso degli eventi, le precondizioni necessarie e i possibili scenari di errore.

Tabella 2.1: Caso d'Uso: Ricarica Veicolo

Campo	Contenuto
Nome	Ricarica Veicolo (Charge Vehicle)
Livello	Obiettivo Utente (User Goal)
Descrizione	Il Conducente avvia una sessione di ricarica su una stazione selezionata, scegliendo una specifica strategia di ricarica (Fast o Eco). La sessione viene eseguita in tempo reale: il Grid Monitor la fa avanzare e l'interfaccia utente (Live Session UI) ne osserva lo stato tramite polling.
Pre-condizioni	Il Conducente ha effettuato l'accesso. Il saldo del Portafoglio (Wallet) del Conducente copre almeno il costo di 5 kWh presso la stazione selezionata, calcolato per il piano del Conducente (<code>station.priceFor(5.0, driver)</code>) e la strategia selezionata. La stazione selezionata è ACTIVE (oppure RESERVED dal Conducente) ed è compatibile con il connettore del veicolo.
Post-condizioni	La sessione è COMPLETED , il costo (detratto in modo incrementale per ogni tick durante la ricarica) è interamente addebitato sul Portafoglio, viene registrata una Transazione di tipo CHARGE e la stazione torna allo stato ACTIVE .
Attori	Conducente, Grid Monitor (Sistema)
Unit Test	<code>SessionServiceTest</code>

Continua nella pagina successiva...

Tabella 2.1 – Continua dalla pagina precedente

Campo	Contenuto
Flusso Principale	1. Il Conducente seleziona una stazione dalla lista delle stazioni disponibili e compatibili più vicine (ACTIVE, o RESERVED da lui stesso). 2. Il Conducente apre la Live Session per quella stazione. 3. Il Conducente sceglie una strategia di ricarica: Fast Charge o Eco Charge. 4. Il Conducente inserisce la percentuale di batteria attuale del veicolo. 5. Il Conducente preme Start; il sistema valida il saldo del Portafoglio e la disponibilità della stazione, imposta la stazione su BUSY e apre la sessione. 6. Il Grid Monitor, ogni 5s, aggiorna batteria e kWh erogati, accumula il calore sul trasformatore di alimentazione e deduce i fondi in base alla strategia (lo sconto del piano si applica solo alla quota tariffaria della piattaforma). 7. La Live Session UI interroga (poll) lo stato della sessione e riflette in tempo reale batteria, kWh, costo attuale e Portafoglio. 8. Il Conducente preme Stop (oppure la sessione si auto-completa quando la batteria raggiunge il 100%). 9. Il sistema calcola i kWh totali erogati, il costo finale e registra la Transazione. 10. Il sistema reimposta lo stato della stazione su ACTIVE e mostra il popup di valutazione (rating).
Flussi Alternativi	1. Stazioni non disponibili nascoste: 1.1. Le stazioni BUSY, OVERLOADED, incompatibili o RESERVED da altri non sono mostrate in lista, impedendo al Conducente di selezionare una stazione non disponibile. 5. Saldo insufficiente: 5.1. Se il saldo è inferiore al costo di 5 kWh, il sistema nega l'avvio e mostra: "Credito insufficiente per avviare la sessione." (<code>InsufficientBalanceException</code>). 6-7. Allarme Termico (Observer attivato): 6.1. Se la temperatura del trasformatore supera i 90°C, il <code>LoadBalancerService</code> (Observer) imposta tutte le stazioni su quel trasformatore su OVERLOADED e chiude forzatamente le relative sessioni attive. 6.2. La sessione diventa INTERRUPTED_THERMAL e la UI mostra: "Arresto di emergenza: surriscaldamento del sistema". 6.3. Il sistema elabora un rimborso automatico per i kWh pagati ma non erogati.

Campo	Contenuto
Nome	Prenotazione a Breve Termine (Short-Term Hold)
Livello	Obiettivo Utente (User Goal)
Descrizione	Il Conducente prenota temporaneamente una stazione di ricarica ACTIVE per 15 minuti per assicurarne la disponibilità al suo arrivo fisico.
Pre-condizioni	• Il Conducente ha effettuato l'accesso. • Il Conducente non ha già un'altra prenotazione attiva. • La stazione di destinazione è ACTIVE ed è compatibile con il connettore del Conducente.
Post-condizioni	Lo stato della stazione viene modificato in RESERVED , reserved_by_id viene impostato sul Conducente e expiration_timestamp viene impostato all'orario attuale più 15 minuti. La stazione prenotata rimane visibile al Conducente che ha effettuato la prenotazione (contrassegnata come RESERVED) e viene nascosta a tutti gli altri Conducenti.
Attori	Conducente
Unit Test	StationServiceTest
Flusso Principale	1. Il Conducente seleziona una stazione ACTIVE dalla lista delle stazioni disponibili. 2. Il Conducente preme Smart Book per confermare la prenotazione. 3. Il sistema verifica che la stazione sia ancora ACTIVE (controllo di concorrenza). 4. Il sistema imposta lo stato della stazione su RESERVED e reserved_by_id sul Conducente. 5. Il sistema imposta expiration_timestamp all'orario attuale più 15 minuti. 6. Il sistema mostra un messaggio di successo; la stazione appare ora come RESERVED nella lista del Conducente, offrendo solo l'opzione Charge Now.
Flussi Alternativi	3. Stazione non disponibile (Conflitto di concorrenza): 3.1. Se la stazione è stata prenotata o occupata da un altro utente nel frattempo, il sistema nega la prenotazione. 3.2. Il sistema mostra il messaggio: "Stazione non più disponibile". 3.3. Ritorno al passo 1. Nota sulla Scadenza: • Se trascorrono i 15 minuti senza che il Conducente avvii una sessione, il Grid Monitor ripristina automaticamente la stazione su ACTIVE (gestito dal caso d'uso Expire Short-Term Holds).

Tabella 2.2: Caso d'Uso: Prenotazione a Breve Termine

2.3 Mockups

In questa fase sono stati realizzati i mockup dell'interfaccia grafica per verificare la User Experience e garantire che tutte le funzionalità descritte nei casi d'uso siano accessibili in modo immediato ed intuitivo.

2.4 Page Navigation Diagram

Il diagramma di navigazione illustra in che modo le diverse schermate dell'applicazione sono collegate tra loro. Mostra il percorso che l'utente compie muovendosi all'interno del sistema in base alle scelte effettuate o ai pulsanti premuti.

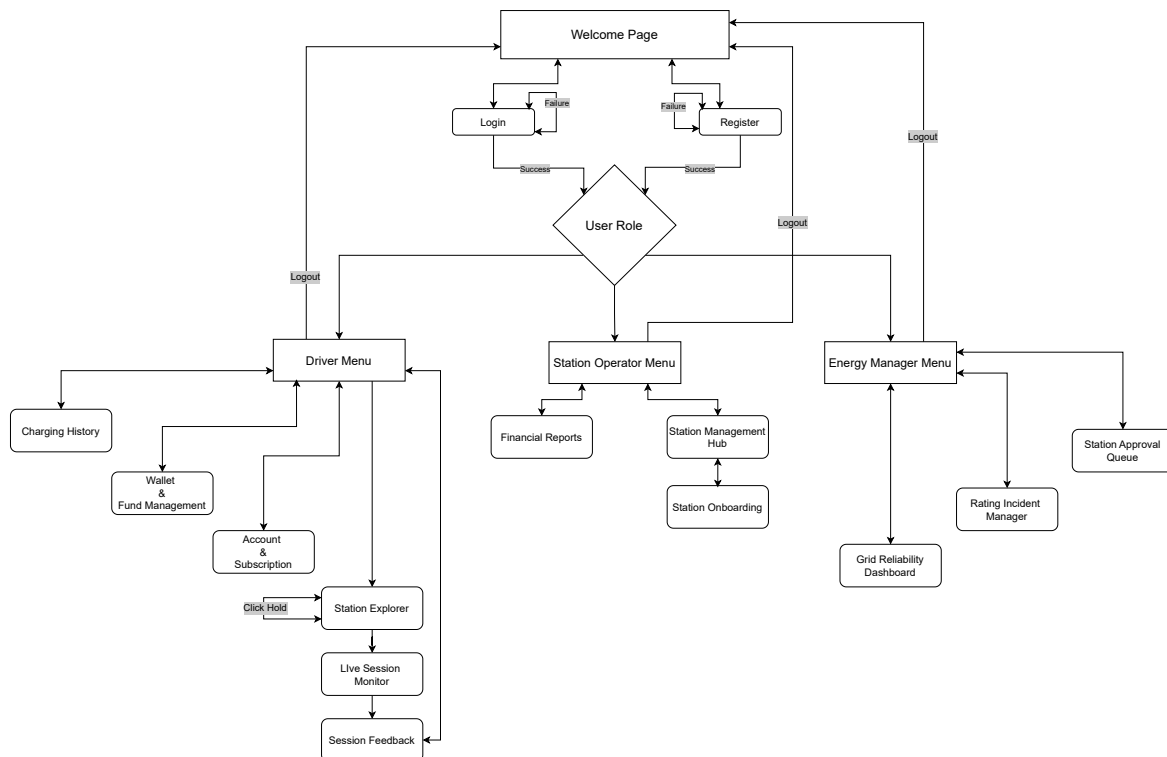


Figura 2.2: Diagramma di navigazione tra le pagine dell'applicazione.

2.5 Package Diagram

Passando all'architettura interna del software, il diagramma dei pacchetti mostra come il codice è stato diviso in moduli logici (come ad esempio la logica di business, l'interfaccia e l'accesso ai dati). L'obiettivo è mantenere il progetto ordinato e limitare le dipendenze tra i vari componenti.

2.6 Class Diagram

Il sistema è stato progettato con un'architettura a strati per mantenere alta coesione e basso accoppiamento (*high coesion and low coupling*). Nel seguente paragrafo sono approfondite le classi principali:

2.7 ER Diagram e Modello Relazionale

L'ultimo passo della progettazione riguarda la persistenza dei dati. In questa sezione viene presentato lo schema Entità-Relazione (ER) del database e la sua successiva traduzione nel modello relazionale a tabelle, pronto per l'implementazione su PostgreSQL.

Capitolo 3

Implementazione

Questo capitolo descrive le scelte implementative e l'architettura interna di ChargeNet. Per mantenere la trattazione chiara, non verrà mostrato l'intero codice sorgente, ma ci si concentrerà sull'organizzazione dei componenti principali e sui frammenti di codice che illustrano le logiche più complesse e i pattern utilizzati.

Architettura del Sistema: I livelli principali sono tre: Domain, Business e Controller. L'interazione con il database PostgreSQL è gestita tramite il pattern Data Access Object (DAO), che nasconde la complessità delle query SQL al resto dell'applicazione.

3.1 Domain Model

Per avere maggiore ordine abbiamo suddiviso il Domain Model in cinque pacchetti principali, ognuno con una responsabilità specifica, raggruppando il codice in base alla finalità e al contesto di utilizzo. Questa struttura modulare facilita la manutenzione e l'estensione del sistema, permettendo di isolare le modifiche a specifici ambiti senza impattare l'intero progetto, oltre che a rendere più semplice la comprensione del codice per il lettore.

Di seguito sono elencati i cinque pacchetti principali, con una breve descrizione delle loro responsabilità, che verranno approfondite nel proseguo del paragrafo:

- **Users:** Definisce gli attori del sistema. Ci sono tre profili principali (*Driver*, *StationOperator* ed *EnergyManager*), che estendono tutti una classe madre comune per semplificare le operazioni di login e registrazione.
- **Infrastructure:** Rappresenta le parti fisiche della rete. Contiene le colonnine (*ChargingStation*) e i trasformatori (*PowerTransformer*), e serve a gestire parametri come la potenza massima erogabile o lo stato di occupazione.
- **Sessions:** È il collegamento pratico tra l'utente e l'infrastruttura. La classe centrale è *ChargingSession*, che si occupa di far partire la ricarica e di calcolare quanti kWh vengono consumati col passare del tempo.
- **Financials:** Si occupa dei pagamenti e del saldo degli utenti. Gestisce le ricevute (*Transaction*) create dopo una ricarica, un rimborso o il pagamento di un abbonamento, facendo in modo che lo storico contabile non possa essere modificato per sbaglio.

- **Feedback:** Gestisce le recensioni. Raccoglie i voti (*Rating*) lasciati dai guidatori e fa scattare in automatico un avviso di manutenzione (*RatingAlert*) se la media di una stazione si abbassa troppo.

Nei prossimi paragrafi vedremo più nel dettaglio alcune delle soluzioni implementative che abbiamo adottato in questi cinque moduli.

3.1.1 Utenti

Questo modulo definisce le identità degli attori tramite un'architettura basata sull'ereditarietà. Dalla classe astratta `User` derivano i tre profili specifici (`Driver`, `StationOperator` ed `EnergyManager`), ognuno con i propri attributi.

Per rispettare il principio dell'incapsulamento, la logica di business è integrata direttamente nelle entità. Un esempio pratico è la gestione economica del `Driver`: quando c'è da scalare del credito, è l'oggetto stesso a fare i controlli sul proprio saldo.

```
public void charge(BigDecimal amount) {
    if (!hasSufficientBalance(amount)) {
        throw new InsufficientBalanceException("Saldo
            insufficiente");
    }
    this.walletBalance = this.walletBalance.subtract(amount);
}
```

Questo approccio ci ha permesso di riutilizzare la validazione di base per operazioni più complesse. Ad esempio, il metodo per cambiare il piano in abbonamento richiama semplicemente il pagamento interno, mantenendo il codice pulito ed evitando duplicazioni:

```
public void updatePlan(SubscriptionPlan newPlan) {
    BigDecimal fee = newPlan.getMonthlyFee();
    if (fee.compareTo(BigDecimal.ZERO) > 0) {
        charge(fee); // Se non ci sono soldi, il lancio dell'
                    eccezione blocca tutto
    }
    this.subscriptionPlan = newPlan;
}
```

Per il collegamento con il database, invece di passare per i costruttori standard (che farebbero scattare validazioni inutili in fase di caricamento), abbiamo dotato ogni entità di un metodo `reconstitute()`, pensato esclusivamente per ripristinare in memoria i dati letti da PostgreSQL.

3.1.2 Infrastrutture

L'obiettivo di questo pacchetto è tradurre in codice i limiti fisici del mondo reale. Per la classe `ChargingStation` era fondamentale impedire la creazione di oggetti con parametri incoerenti o pericolosi.

Abbiamo quindi nascosto i costruttori, obbligando il resto del sistema a passare per il Factory Method `register()`. In questa fase la stazione fa un vero e proprio controllo

di sicurezza: blocca sul nascere eventuali richieste assurde, come ad esempio tentare di associare una potenza di 350 kW a un connettore che fisicamente ne regge al massimo 22.

```
public static ChargingStation register(StationOperator operator,
    PowerTransformer transformer, Double lat, Double lng,
    ConnectorType connectorType, Double powerKw, BigDecimal tariff)
{
    if (powerKw > connectorType.getMaxPowerKw()) {
        throw new InvalidStationParametersException(
            "Errore di sicurezza: La potenza richiesta supera il
            limite fisico del connettore.");
    }
    return new ChargingStation(operator, transformer, lat, lng,
        connectorType, powerKw, tariff);
}
```

L'altro parte fondamentale del pacchetto è il **PowerTransformer**, che fa da base per il sistema di Load Balancing. Piuttosto che avere un servizio esterno che controlla continuamente i trasformatori, abbiamo reso il trasformatore intelligente applicando il pattern Observer. A ogni ciclo di simulazione, ricalcola la sua temperatura interna; se supera la soglia critica dei 90 gradi, genera autonomamente un **THERMAL_ALERT** per avvisare la rete, senza bisogno di sapere chi o come gestirà effettivamente l'emergenza.

3.1.3 Sessioni

Il pacchetto racchiude la logica legata all'erogazione dell'energia. La classe centrale, **ChargingSession**, non si limita a registrare passivamente i dati (come kWh erogati e costi parziali), ma regola attivamente l'intero ciclo di vita dell'operazione, dalla creazione fino al completamento naturale o all'interruzione per emergenza termica.

Per garantire che una ricarica non inizi in condizioni non valide, l'istanziamento è affidata al metodo factory **open()**. Questo metodo esegue una serie di controlli incrociati di dominio: verifica la compatibilità fisica del connettore, controlla che la stazione non sia occupata e calcola se il conducente possiede un saldo minimo sufficiente, tenendo conto anche dei moltiplicatori di sconto legati al suo piano di abbonamento:

```

public static ChargingSession open(Driver driver, ChargingStation
    station, ChargingType strategy, Double batteryStart) {
    boolean isAvailable = station.isAvailableFor(driver.
        getConnectorType()) ||
        station.isReservedBy(driver);
    if (!isAvailable) {
        throw new StationNotAvailableException("Colonnina non
            compatibile o occupata.");
    }

    BigDecimal pricePerKwh = station.getTotalTariff();
    BigDecimal discountMult = BigDecimal.ONE.subtract(BigDecimal.
        valueOf(driver.getDiscount()));
    BigDecimal minimumRequired = pricePerKwh.multiply(BigDecimal.
        valueOf(5)).multiply(discountMult);

    if (!driver.hasSufficientBalance(minimumRequired)) {
        throw new InsufficientBalanceException("Credito
            insufficiente per avviare la sessione.");
    }

    return new ChargingSession(driver, station, strategy,
        batteryStart);
}

```

Inoltre, la classe incapsula la logica di avanzamento tramite il metodo `addTick()`, che aggiorna in tempo reale i costi e l'energia immessa, facendo transitare automaticamente lo stato della sessione a completato qualora la batteria raggiunga il 100% della sua capacità.

3.1.4 Finanze

Questo modulo gestisce lo storico dei movimenti economici degli utenti, categorizzandoli tramite l'enumerazione `TransactionType` (ricariche, rimborsi, abbonamenti e depositi).

La scelta architetturale più rilevante per l'entità `Transaction` è stata quella di renderla intrinsecamente immutabile. Trattandosi di dati contabili, era fondamentale garantire che uno storico non potesse essere alterato accidentalmente a runtime. Per questo motivo, la classe è sprovvista di metodi *setter* e i suoi campi sono valorizzabili unicamente al momento della creazione.

Per blindare ulteriormente l'integrità del registro, la generazione di un nuovo movimento non avviene tramite un normale costruttore, ma è protetta da un metodo *factory* che scarta a priori qualsiasi operazione anomala, come ad esempio i pagamenti a importo zero o privi di causale:

```

public static Transaction create(Driver driver, TransactionType
    type, BigDecimal amount,
                                Double kwh, String description)
    {

        if (amount == null || amount.compareTo(BigDecimal.ZERO) == 0)
        {
            throw new IllegalArgumentException("Errore: l'importo non
                puo' essere zero.");
        }
        if (description == null || description.trim().isEmpty()) {
            throw new IllegalArgumentException("Errore: la
                transazione deve avere una descrizione.");
        }
        return new Transaction(driver, type, amount, kwh, description
            );
    }

```

Questa impostazione si è rivelata molto utile per semplificare la logica del livello Business, sapendo di poter fare affidamento su un tracciamento finanziario solido e privo di "transazioni fantasma".

3.1.5 Feedback

A chiudere il Domain Model troviamo la parte dedicata alla qualità del servizio e al supporto. Qui abbiamo deciso di separare l'azione dell'utente (la recensione) dalla conseguente gestione amministrativa (l'allarme per la manutenzione).

Nella classe `Rating`, la creazione di un nuovo feedback è protetta da controlli rigorosi. Abbiamo implementato il metodo `factory leave()`, che non si limita a istanziare la recensione, ma lega quest'ultima alla sessione di ricarica effettiva e impedisce l'inserimento di voti fuori dal range previsto:

```

public static Rating leave(Driver driver, ChargingStation station
    , ChargingSession session, Integer stars, String comment) {
    if (stars == null || stars < 1 || stars > 5) {
        throw new InvalidRatingException("Errore di validazione:
            il rating deve essere un valore compreso tra 1 e 5
            stelle.");
    }
    return new Rating(driver, station, session, stars, comment);
}

```

Il vero aspetto interessante di questo modulo, tuttavia, si trova nella classe `RatingAlert`. Quando la media di una stazione cala drasticamente, il sistema genera un avviso. Invece di affidare la chiusura di questo ticket a un servizio esterno, abbiamo inserito la logica di transizione di stato direttamente all'interno dell'entità. I metodi per sospendere la colonnina o ignorare l'allarme verificano prima che la segnalazione sia effettivamente ancora in sospeso, evitando che un operatore possa inavvertitamente alterare pratiche già chiuse:


```

public void resolveSuspend(String note) {
    if (isPending()) {
        this.status = RatingAlertStatus.RESOLVED_SUSPENDED;
        this.managerNote = note;
    }
}

```

3.2 Business Layer

Il Business Layer rappresenta il livello in cui i dati del Domain Model vengono elaborati e trasformati in funzionalità concrete. L'architettura di questo strato è stata progettata seguendo il principio di singola responsabilità (SRP), suddividendo il codice in servizi specializzati (*Services*) e componenti di simulazione (*Core*).

L'implementazione delle diverse logiche di erogazione (*ChargingStrategy*) verrà approfondita nella successiva sezione dedicata ai Design Pattern.

Di seguito vengono analizzate nel dettaglio le classi centrali che governano e dirigono il flusso dell'applicazione.

3.2.1 GridCluster

Il primo problema tecnico che abbiamo affrontato progettando la simulazione è stato il carico sul database. Poiché il sistema deve ricalcolare i consumi e le temperature della rete ogni pochi secondi, eseguire continuamente query di lettura tramite i DAO avrebbe rallentato l'intera applicazione.

Per risolvere questo collo di bottiglia, abbiamo implementato **GridCluster**. Si tratta di una cache in memoria, gestita come Singleton, che viene popolata una sola volta all'avvio dell'applicazione. Al suo interno, i dati non sono salvati in semplici liste, ma indicizzati strategicamente tramite mappe Hash per garantire tempi di accesso costanti ($O(1)$). Ad esempio, raggruppando le colonnine per ID del trasformatore, il sistema può recuperare l'intero ramo di rete istantaneamente:

```

// Struttura dati per accesso rapido in memoria
private Map<Long, List<ChargingStation>> stationsByTransformer;

public void init(TransformerDao tDao, StationDao sDao) {
    this.transformers = tDao.findAll();
    this.allStations = sDao.findAll();

    for (PowerTransformer t : transformers) {
        stationsByTransformer.put(t.getId(), new ArrayList<>());
    }
    for (ChargingStation station : allStations) {
        stationsByTransformer.get(station.getTransformer().getId())
            .add(station);
    }
}

```

Oltre alla gestione della rete elettrica, questa classe espone anche i metodi di ricerca geografica, utilizzando la distanza euclidea per restituire rapidamente all'utente le colonnine disponibili più vicine.

3.2.2 GridMonitor

Non avendo a disposizione dei sensori fisici che inviano dati telemetrici, avevamo bisogno di un componente che facesse scorrere il tempo in modo autonomo, senza bloccare l'interfaccia grafica.

Abbiamo quindi esteso la classe `Thread` di Java. Un dettaglio implementativo fondamentale è stato impostare questo thread come "Demone" all'interno del costruttore (`this.setDaemon(true);`). In questo modo, il ciclo vitale del monitor viene legato a quello dell'interfaccia JavaFX: se l'utente chiude la finestra dell'applicazione, il thread si arresta in modo pulito senza lasciare processi appesi in background.

Il ciclo di esecuzione principale si sveglia ogni 5 secondi e orchestra le operazioni dei vari servizi, aggregando i dati prima di applicarli:

```
private void onTick() {
    // 1. Libera le prenotazioni scadute
    expireShortTermHolds();

    // 2. Calcola la mappa del calore generato dalle auto in
    // carica
    Map<PowerTransformer, Double> heatMap = processActiveSessions
        ();

    // 3. Applica il calore calcolato ai trasformatori
    processTransformers(heatMap);

    // 4. Verifica eventuali crolli nelle recensioni
    checkRatingAlerts();
}
```

In particolare, la fase di elaborazione (`processActiveSessions`) calcola il calore generato dalle singole strategie di ricarica in uso (Fast o Eco) e lo somma all'interno di una *Heat Map*, in modo da notificare i trasformatori una sola volta per ciclo, ottimizzando ulteriormente le prestazioni.

3.2.3 LoadBalancerService

Il `LoadBalancerService` è il componente che giustifica l'esistenza del pattern Observer all'interno dell'architettura. Invece di interrogare attivamente la rete per scovare eventuali surriscaldamenti, questo servizio si registra semplicemente in ascolto sui trasformatori.

Quando riceve una notifica di `THERMAL_ALERT`, il servizio reagisce isolando immediatamente il ramo di rete compromesso. L'intervento si divide in due fasi distinte per evitare *deadlock* sul database. Prima viene aperta una transazione esclusiva per forzare lo stato di tutte le colonnine collegate a `OVERLOADED`, impedendo l'avvio di nuove ricariche. Successivamente, la transazione viene chiusa e il servizio delega l'interruzione fisica delle erogazioni già in corso al modulo competente.

```

private void handleThermalAlert(PowerTransformer transformer) {
    // 1. Aggiornamento massivo e atomico delle colonnine
    Connection connection = null;
    try {
        connection = dbManager.getConnection();
        connection.setAutoCommit(false);
        StationDao stationDao = daoFactory.createStationDao(
            connection);

        for (ChargingStation station : stations) {
            station.setOverloaded();
            stationDao.update(station);
        }
        connection.commit();
    } finally {
        closeQuietly(connection); // Previene blocchi a cascata
    }

    // 2. Interruzione forzata delle sessioni in corso
    for (ChargingSession session : sessionService.
        getActiveSessions()) {
        if (belongsToTransformer(session.getStation(), stations))
        {
            sessionService.forceClose(session);
        }
    }
}
}

```

3.2.4 SessionService

Mentre il Load Balancer gestisce le emergenze, il **SessionService** coordina la normale operatività economica ed energetica. La sfida principale in questa classe è stata la corretta gestione del *Commit* a due fasi.

Ogni volta che una ricarica avanza (tramite il metodo `addTick()`) o viene interrotta, il sistema deve aggiornare contemporaneamente entità diverse: la sessione vera e propria, lo stato della colonnina e il portafoglio dell'utente.

Invece di delegare la gestione delle connessioni SQL ai singoli DAO, abbiamo centralizzato il controllo transazionale all'interno del Service. Modificando l'impostazione predefinita della connessione (`setAutoCommit(false)`), garantiamo l'atomicità dell'operazione. Se durante il salvataggio dei dati energetici si verifica un errore nel prelievo del denaro dal *Wallet*, l'intera operazione viene annullata tramite `rollback`, assicurando che l'utente non venga mai disallineato rispetto alla colonnina fisica.

Questa scelta progettuale, applicata trasversalmente a tutti i metodi del livello Business (da `openSession` a `forceClose`), ha reso l'architettura estremamente resiliente ai fallimenti hardware o di rete simulati.

3.2.5 StationService

Sebbene gran parte di questo servizio si occupi di gestire il salvataggio e la validazione delle nuove colonnine (delegando i controlli di sicurezza ai Factory Method del dominio), è presente una complessità notevole nella gestione delle prenotazioni (*Short-Term Hold*).

Infatti il problema principale in fase di prenotazione è la concorrenza: due Driver potrebbero tentare di riservare la stessa stazione nell'esatto momento in cui si libera. Per prevenire *race condition*, il servizio non si limita a leggere lo stato della colonnina e aggiornarlo in memoria, ma sfrutta i lock del database. Richiamando un'operazione di aggiornamento atomico tramite il DAO (`acquireAtomicHold`), il servizio garantisce che solo la prima transazione vada a buon fine, bloccando in modo sicuro e definitivo la risorsa e lanciando una `StationNotAvailableException` per il secondo utente.

3.2.6 Sicurezza e Validazione

La responsabilità di sicurezza dell'applicazione è divisa in due aree distinte, gestite da servizi separati:

- **Autenticazione (AuthService):** Il sistema non salva mai le credenziali in chiaro. L'hashing delle password è delegato alla libreria esterna **BCrypt**. Inoltre, il flusso di registrazione varia dinamicamente in base al ruolo richiesto, imponendo l'assegnazione di coordinate GPS iniziali esclusivamente ai Driver.
- **Collaudo dell'Infrastruttura (ValidationService):** Nessuna stazione registrata da un operatore entra in rete automaticamente. Prima dell'approvazione, l'Energy Manager avvia un collaudo software per intercettare anomalie hardware.

È proprio quest'ultimo punto a evitare gli errori di inserimento. Ecco come il sistema "interroga" la colonnina prima di permetterne l'attivazione:

```
public boolean testStation(ChargingStation station) {
    // 1. Controllo di coerenza fisica
    if (station.getPowerKw() > station.getConnectorType().
        getMaxPowerKw()) {
        System.err.println("Test fallito: Potenza eccessiva per
            il tipo di connettore.");
        return false;
    }

    // 2. Simulazione del ping di rete
    return simulateConnectivityTest(station.getLatitude(),
        station.getLongitude());
}
```

Solo se il metodo restituisce `true`, il servizio procede ad aprire la transazione sul database, fissando la tariffa della piattaforma e cambiando lo stato della stazione in `ACTIVE`.

3.2.7 Gestione Utente e Portafoglio

La gestione del profilo utente e del suo portafoglio è affidata al `DriverService` e al `WalletService`. Quest'ultimo, in particolare, implementa le funzionalità legate ai pagamenti e ai piani tariffari, sollevando la classe di dominio `Driver` dalla responsabilità di interfacciarsi con il database.

Una delle operazioni più delicate di questo modulo è l'aggiornamento dell'abbonamento mensile. Per garantire che l'utente non generi debiti non autorizzati, l'attivazione di un piano a pagamento (es. PLUS o PREMIUM) richiede un controllo preventivo sul saldo corrente. Se i fondi sono sufficienti, il servizio orchestra due azioni distinte: delega l'aggiornamento del moltiplicatore di sconto all'oggetto `Driver` e genera simultaneamente una ricevuta per tenere traccia dello storno.

Anche in questo caso, per mantenere la coerenza del registro contabile, l'aggiornamento del profilo e il salvataggio della ricevuta avvengono all'interno di un unico blocco transazionale, garantendo che un eventuale errore nel salvataggio della `Transaction` annulli istantaneamente anche le modifiche apportate in memoria all'utente.

```
public boolean testStation(ChargingStation station) {
    // 1. Controllo di coerenza hardware
    if (station.getPowerKw() > station.getConnectorType().
        getMaxPowerKw()) {
        System.err.println("Test fallito: Potenza eccessiva per
            il tipo di connettore.");
        return false;
    }

    // 2. Simulazione test di connettività
    boolean pingSuccess = simulateConnectivityTest(station.
        getLatitude(), station.getLongitude());
    if (!pingSuccess) {
        System.err.println("Test fallito: Impossibile stabilire
            la connessione con la colonnina.");
        return false;
    }

    return true;
}
```

Solo se questo test ha esito positivo, il servizio permette di sbloccare l'operazione di `approve`, aprendo una transazione che cambia lo stato della stazione e fissa in modo definitivo la tariffa trattenuta dalla piattaforma.

3.2.8 Gestione Feedback e Manutenzione

Il flusso di controllo della qualità del servizio è interamente gestito dal `RatingService`. Il ciclo di vita di un feedback inizia nel momento in cui il `Driver` conclude una ricarica e decide di valutare l'esperienza.

In questa prima fase, il sistema esegue un duplice controllo. Dapprima verifica a livello database che l'utente non abbia già recensito quella specifica sessione; superato questo filtro, delega la validazione del formato del voto (da 1 a 5 stelle) al dominio.

La parte interessante e funzione aggiuntiva di questa componente risiede nella logica che si innesca subito dopo il salvataggio. Infatti, non essendoci un controllo umano costante sulle singole colonnine, il servizio ricalcola in tempo reale la media della stazione appena recensita. Se tale parametro scende sotto la soglia critica di 2.0 (con un campione minimo di 5 valutazioni), il sistema interviene autonomamente generando un `RatingAlert`.

```
private void checkRatingAlert(Connection connection,
    ChargingStation station) {
    // Ricalcolo della media tramite query aggregata SQL
    ratingDao.recalculateAverage(station.getId());
    ChargingStation updatedStation = stationDao.findById(station.
        getId());

    Double currentAvg = updatedStation.getAverageRating();
    Integer totalRatings = updatedStation.getTotalRatings();

    // Regola di Business per l'apertura del ticket
    if (currentAvg != null && currentAvg < 2.0 && totalRatings !=
        null && totalRatings >= 5) {
        if (!ratingAlertDao.existsOpenAlertForStation(station.
            getId())) {
            RatingAlert alert = RatingAlert.create(updatedStation
                , currentAvg);
            ratingAlertDao.save(alert);
        }
    }
}
```

Una volta ricevuta la segnalazione sulla propria dashboard, sarà discrezione dell'Energy Manager decidere se ignorarla (qualora si trattasse di un falso allarme) o sospendere la stazione difettosa.

Nota: la mappa di Firenze e il Jitter

Per rendere i test più verosimili, abbiamo deciso di utilizzare come ambiente geografico l'area urbana di Firenze, mappando i veri quartieri (dal Centro Storico alla periferia) tramite l'enumerazione `Zone`.

Ci siamo però accorti subito di un problema: registrando più colonnine nello stesso quartiere, queste avrebbero assunto le stesse identiche coordinate di base, finendo per sovrapporsi perfettamente in caso di futura rappresentazione grafica. Per risolverlo, abbiamo introdotto una piccola funzione di *Jitter* all'interno dell'enum. Questo metodo calcola un microscopico scostamento casuale (corrispondente a circa 500 metri) e lo applica al centro esatto della zona, in modo tale da generare latitudini e longitudini leggermente diverse per ogni stazione, pur rimanendo all'interno del quartiere di riferimento.

```
public double[] resolveWithJitter() {  
    // +/- 0.005 gradi di variazione casuale su latitudine e  
    // longitudine  
    double jLat = (Math.random() - 0.5) * 0.01;  
    double jLng = (Math.random() - 0.5) * 0.01;  
    return new double[]{ baseLat + jLat, baseLng + jLng };  
}
```

3.3 Controller (Application Layer)

Il livello Controller fa da collante tra l'interfaccia grafica JavaFX e la logica di business. I controller intercettano le azioni dell'utente (come il click su un pulsante per avviare una ricarica), richiamano i servizi appropriati nel Business Layer e aggiornano la vista di conseguenza.

3.4 Design Pattern Implementati

Per risolvere problematiche ricorrenti di progettazione in modo elegante ed estendibile, sono stati adottati diversi Design Pattern. Di seguito vengono analizzate le implementazioni più significative.

Capitolo 4

Testing

Capitolo 5

Demo